

Logical Clocks

Distributed Systems / Case Study

Andrea Omicini

<mailto:andrea.omicini@unibo.it>

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



- 1 Actions, Events, Causes
- 2 Logical Clocks
- 3 Vector Clocks



Next in Line...

1 Actions, Events, Causes

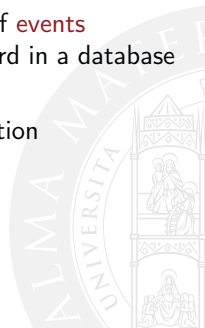
2 Logical Clocks

3 Vector Clocks



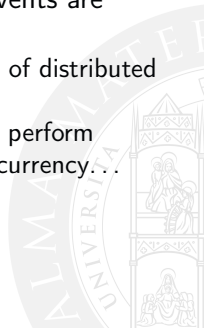
Actions & Events^[Baquero and Preguiça, 2016]

- any computational systems can be modelled as a sequence of executed **actions**
 - with an action being defined by changes in the state of the system, or, more generally, in the world
 - e.g., reading from memory, writing to a file, switching on a lamp
- in a distributed system, actions execute in **multiple locations**
 - in this context, actions are often represented in terms of **events**
 - e.g., sending or receiving messages, changing some record in a database
- *physical distribution* may vary hugely
 - events in a distributed system can occur in a close location
 - e.g., different processes running on the same machine
 - or, geographically spread across the globe
 - or even at a larger scale in the near future



Events & Causes^[Baquero and Preguiça, 2016]

- *not all* events are *related*, but some events can **cause** / influence how other (later) events occur
 - e.g., a reply to a received email message is influenced by that message—maybe by other prior messages received
- detecting potential *cause-and-effect* relations between events are fundamental to the design of *distributed algorithms*
 - nowadays, basically every service is based on some form of distributed algorithm at its core
 - e.g., to maintain consistency in replicated databases, to perform garbage collection, to build checkpoints, to analyse concurrency...



Internal vs. External Causality [Baquero and Preguiça, 2016]

- to make sense of cause-and-effect relations, we should limit their scope to what can be perceived *inside* the distributed system itself
 - internal causality
- yet, a distributed system interacts with the *outside* physical world, where other cause-and-effect relations exist at large
 - external causality
 - e.g., Alice reserves for dinner for two and *tells Bob*, then Bob reserves two movie tickets for the after dinner: even in the case they are part of the same system (e.g., restaurant in a multiplex), the two reservations systems know nothing of the external causal relation (Alice tells Bob)
- external causality cannot be *generally detected* by a distributed system and can possibly be approximated by physical time only
 - if we forget about specific domain knowledge

Next in Line...

- 1 Actions, Events, Causes
- 2 Logical Clocks
- 3 Vector Clocks

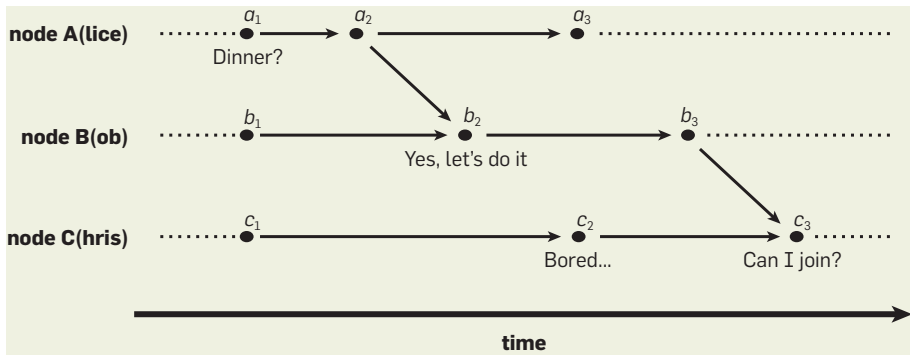


Detecting Causality I

- in order to detect *potential* causality relations, we need some sort of temporal relationship between otherwise-unrelated events in a distributed system
- we start from the notion that event c can **cause** event e *if and only if*
 - with c, e occurring in the same process, c executed first, or
 - with c, e occurring in different processes, e could know about c based on some message received from some process knowing about c
- otherwise, no way c could work as a cause for e



Detecting Causality II



Happens-before relation [Baquero and Preguiça, 2016]

Relation *happens-before* [Lamport, 1978]

- let us consider generic events a , b , c
- $a \rightarrow b$ represents the *relation happens before*, “ a happens before b ”
- it means that (all processes agree that) a occurs first, then b occurs
- $a \rightarrow b$ can be directly observed in two situations
 - 1 if a and b are events of the same process, and a comes before b , then $a \rightarrow b$ — local events are ordered by local time
 - 2 if a message is sent by process with an event a , and received by another process with an event b , then $a \rightarrow b$ — a message takes a finite, positive, non-zero amount of time to propagate from sender to receiver
- $a \rightarrow b$ is a *transitive* relation: $a \rightarrow b$, $b \rightarrow c$ imply $a \rightarrow c$
- *happens-before* defines a *partial ordering* over the events in a distributed system
 - when neither $a \rightarrow b$ nor $b \rightarrow a$ can be observed, then nothing can be said on their ordering
 - a and b are said to be *concurrent*: $a \parallel b$

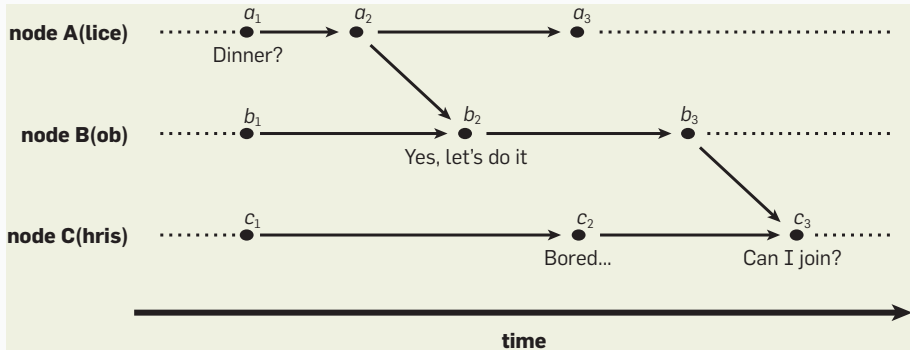
Causal Paths I

- in a distributed computation, a simple way to check if an event c could have caused another event e is to find at least one **directed path** from c to e
 - based on the *happens-before* relation
 - aptly defined as a transitive relation
- if such a directed path is found, c is a possible cause for e
 - $c \rightarrow e$



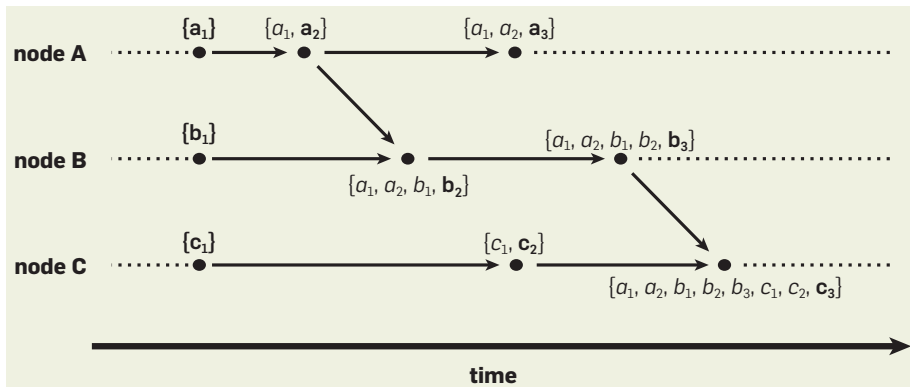
Causal Paths II

In the example...



- $a_1 \rightarrow a_2$, $a_1 \rightarrow b_2$, $b_2 \rightarrow c_3$, $a_1 \rightarrow c_3$, ...
- a_1 and c_2 are instead concurrent ($a_1 \parallel c_2$), because there are no causal paths in either direction

Causal Histories



Causal histories [Baquero and Preguiça, 2016]

Time, Causes, Events

- in distributed systems, *time* could work as an approximation for causality
 - something happening before in time might be the cause of something happening after
 - yet, in this way we lose track of any explicit connection between events in distributed processes
 - a message
 - and, a notion of common time in a distributed system might just not be a thing
- ! a **logical** notion of *time* could be of help here



Logical Time I

Connecting time & events: time values, logical clock

- a shared notion of time for an event a : **time value** $C(a)$ is such that *every process agrees* upon it
 - time value should be thought as the value of a *logical clock* upon which processes agree
- more formally, if H is the domain of events, and T the domain of (logical) time, the **logical clock** is a function $C : H \mapsto T$
- such that the following *monotonicity* property is satisfied
 - $\forall a, b \in H, a \rightarrow b \implies C(a) < C(b)$
- ! **clock consistency condition**
- when T and C further satisfy the following condition
 - $\forall a, b \in H, a \rightarrow b \iff C(a) < C(b)$
- the systems of clocks is said to be **strongly consistent**

Logical Time II

Implementing logical clocks

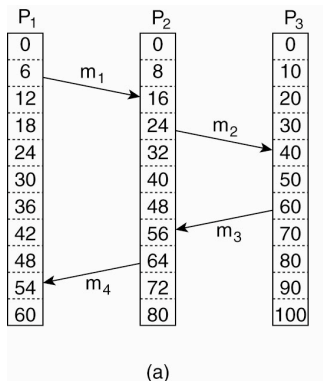
- implementing logical clocks requires addressing two issues
 - ① every process needs local data structures local to represent logical time
 - ② a shared protocol to update the data structures to ensure consistency
- 1 each process p_i maintains a *local* and a *global* logical clock
 - local clock representing the process' progress over time
 - global clock representing the process' view of the global time
- 2 two rules to consistently manage logical clocks, defining how
 - local clock is updated
 - global clock is updated

Logical Time III

Systems of logical clocks^[Kshemkalyani and Singhal, 2011]

- *systems of logical clocks* differ in their representation of logical time and also in the protocol to update the logical clocks
- however, all logical clock systems implement the two aforementioned rules, then ensure the fundamental monotonicity property associated with causality
- moreover, each logical clock system provides for some specific properties
 - e.g., **scalar time**

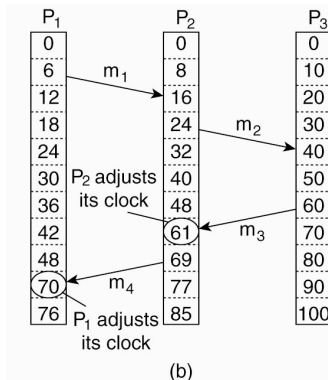
Lamport's Algorithm: Scalar Time I



Distributed message transmission using logical clocks with scalar time

[Tanenbaum and van Steen, 2017]

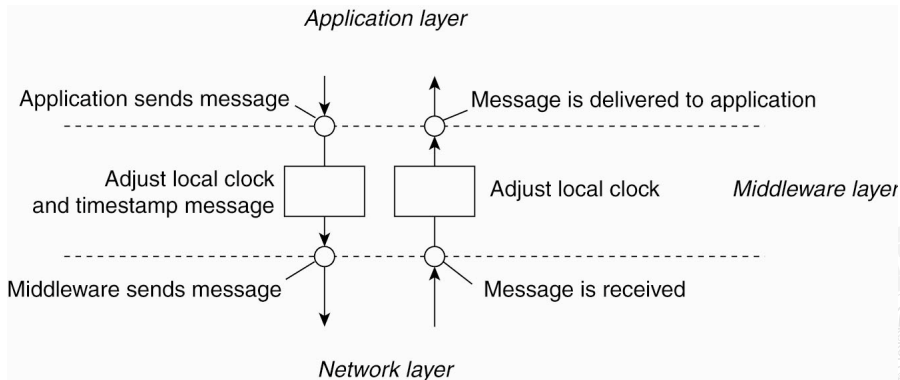
Lamport's Algorithm: Scalar Time II



Correcting logical clocks with Lamport's algorithm

[Tanenbaum and van Steen, 2017]

Lamport's Algorithm: Scalar Time III



Middleware support for Lamport's logical clocks

[Tanenbaum and van Steen, 2017]

Lamport's Algorithm: Scalar Time IV

Implementation of Lamport's logical clocks

- each process p_i maintains a *local clock* lc_i
 - and a local clock increment δ_i
- local clocks are updated following three steps
 - 1 before any local event, p_i increments lc_i
 - $lc_i \leftarrow lc_i + \delta_i$
 - δ_i can be 1, in which case other properties are achieved by the protocol
 - 2 when sending a message m to p_j , process p_i piggybacks m with lc_i after updating it
 - 3 upon reception of a message m , process p_j adjusts its local counter lc_j to be coherent with the value lc_i received as a *timestamp* for m
 - $lc_j \leftarrow \max(lc_j, lc_i)$

Lamport's Algorithm: Scalar Time V

Distributed implementation of global time

- lc_i is local time at process p_i
 - $a \in H$ is an event in a distributed system
 - $\forall i, \forall a \in p_i, C(a) \leftarrow lc_i(a)$
- $C(a)$ is the (*logical*) **global time** for the distributed system

Consistency of scalar clocks

- it is easy to see that scalar clocks are monotonous by construction
- so they satisfy the consistency property
 - $\forall a, b \in H, a \rightarrow b \implies C(a) < C(b)$

Lamport's Algorithm: Scalar Time VI

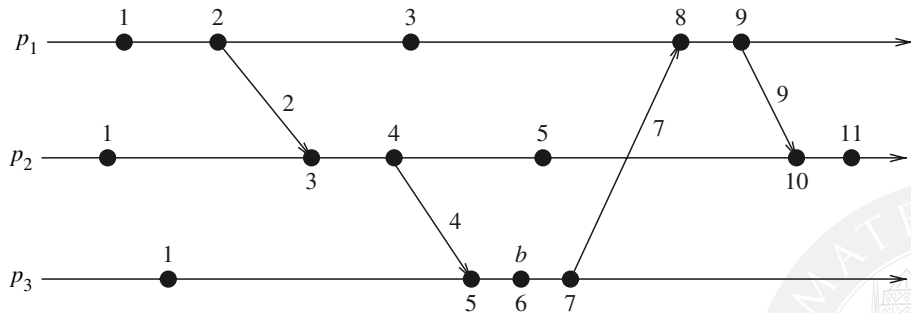


Figure 1: Evolution of scalar time [Kshemkalyani and Singhal, 2011]

Lamport's Algorithm: Scalar Time VII

Total ordering of events^[Kshemkalyani and Singhal, 2011]

- scalar clocks can be used to establish a total ordering relation among events
- by using a tie-breaking mechanism for concurrent events
 - when $a, b \in H, C(a) = C(b)$
- e.g., by providing processes with linearly-ordered identifiers
 - assumption: concurrent events are independent, so no causality relation is violated
 - e.g. in Figure 1, the third event of p_1 has the same timestamp as the second event of p_2 , but can be ordered according to the process identifier
- ! liveness properties typically rely on total ordering, event queues

Lamport's Algorithm: Scalar Time VIII

Event counting^[Kshemkalyani and Singhal, 2011]

- if increment δ is always 1, scalar clocks have the following property
 - if an event a has timestamp h , $h - 1$ represents the *minimum logical duration* in terms of number of events
 - that is, the number of events required to produce event a is $h - 1$
 - called the *height* of the event
 - in other words, $h - 1$ events have been produced sequentially before the event a , regardless of the processes that produced these events
- e.g. in Figure 1, five events precede event of b on the longest causal path ending at b



Lamport's Algorithm: Scalar Time IX

No strong consistency^[Kshemkalyani and Singhal, 2011]

- the system of scalar clocks is not strongly consistent
 - $\forall a, b \in H, C(a) < C(b) \not\Rightarrow a \rightarrow b$
- e.g. in Figure 1, the third event of p_1 has smaller scalar timestamp than the third event of process p_2 , even though the former did not happen before the latter
- this is because the logical local clock and logical global clock of a process are squashed into one, resulting in the loss causal dependency information among events at different processes
- e.g. in Figure 1, when process p_2 receives the first message from process p_1 , it updates its clock to 3, and forgets that the timestamp of the latest event at p_1 on which it depends is 2

Next in Line...

- 1 Actions, Events, Causes
- 2 Logical Clocks
- 3 Vector Clocks**



The Problem

- in essence, $a \rightarrow b$ implies $C(a) < C(b)$
- whereas $C(a) < C(b)$ does not imply $a \rightarrow b$
 - so that, for instance, time values could be totally ordered when events are not
 - when events are unrelated, comparison of time values is meaningless
- scalar clocks say nothing about that
- something more is needed—see causal histories above
- to say in particular whether a and b are (un)related / concurrent



Definition

Vector clocks

- a **vector clock** of a system of n processes is an array/vector of n logical clocks, one clock for each process
 - ! hypothesis: the number n of processes is known
 - ! we may use smaller vectors, but all properties are proven to hold with n
- each process p_i maintains a *vector* vc_i such that
 - $vc_i[i]$ is the number of events occurred so far at p_i
 - basically, the local scalar clock of p_i
 - ← every new event occurring in p_i increments $vc_i[i]$
 - $vc_i[j] = k$ means that p_i knows that (at least) k events have occurred at p_j
 - basically, the scalar clock of p_j according to p_i 's best knowledge
- ← every message from p_i is piggybacked with (current) vector vc_i as a timestamp

Rules

- before any action is executed at p_i , vc_i is updated
 - $vc_i[i] \leftarrow vc_i[i] + 1$
- when a message m is **sent** by p_i to p_j , m is *timestamped* with vector vc_i
 - $ts(m) \leftarrow vc_i$
- when a message m is **received** by p_j , vc_j is *adjusted* accordingly
 - $\forall k, vc_j[k] \leftarrow \max(vc_j[k], ts(m)[k])$
 - if m comes from p_i with timestamp vc_i ,
 $\forall k, vc_j[k] \leftarrow \max(vc_j[k], vc_i[k])$



Example

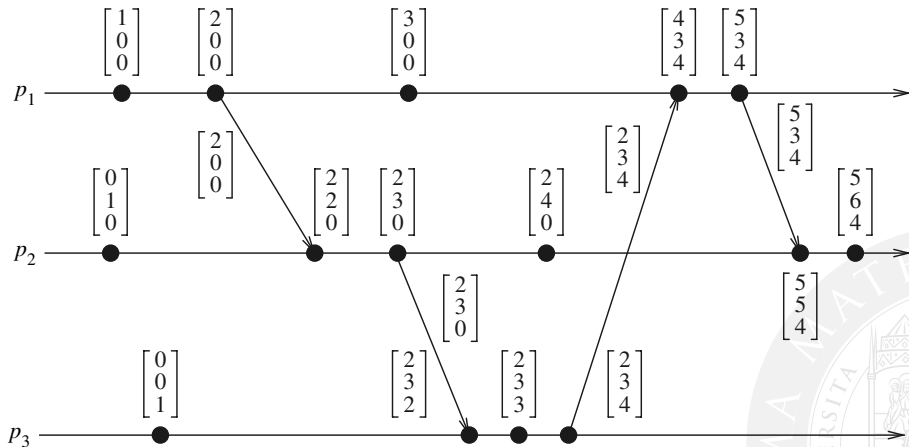


Figure 2: Evolution of vector time [Kshemkalyani and Singhal, 2011]

Ordering

- if we label every event m with $vc(m)$
- and define an *ordering relationship* for vector clocks

$$vc(m_1) < vc(m_2)$$

$$\iff$$

$$\forall i \ vc(m_1)_i \leq vc(m_2)_i \wedge \exists i' \ vc(m_1)_{i'} < vc(m_2)_{i'}$$

- which
 - is antisymmetric and transitive
 - captures simple logical clocks
 - but mostly captures a causal relationship between two events
- such that $vc(m_1) < vc(m_2) \Rightarrow m_1 \rightarrow m_2$
- ! so, vector clocks provide for strong consistency

$$vc(m_1) < vc(m_2) \iff m_1 \rightarrow m_2$$

even though the ordering relation on vc is *per se* not total



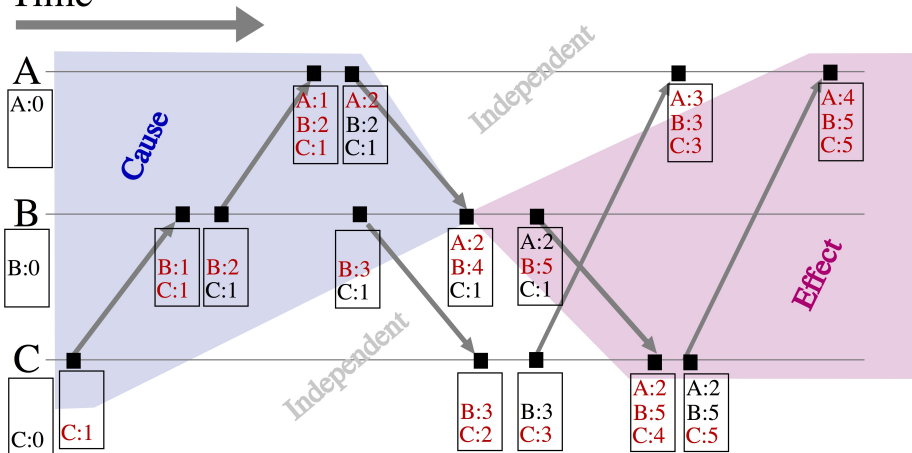
Result

- every process knows how many events have preceded the sending of the received message at the sender process
 - information about the “chain of events” is somehow preserved and shared among processes
- each $ts(m)[i]$ refers to the events *causally preceding* m within each process p_i
- $ts(m)$ keeps track of all the events that may have causally preceded the sending of m , on which m may causally depend



Causality & Vector Clocks

Time



Causality via vector clocks http://wikipedia.org/wiki/Vector_clock

Matrix Time

- and yet, some “squashing” still happens in vector clocks, too
- *matrix clocks* extend logical time further...
- ... but we stop here



Summing Up

- actions and events in distributed systems can be observed under the lens of *causality*
- relation *happens-before* can capture potential causality
- *logical clocks*, *vector clocks*, and *matrix clocks* can provide logical time to a distributed system to account for time ordering and causality



Logical Clocks

Distributed Systems / Case Study

Andrea Omicini

<mailto:andrea.omicini@unibo.it>

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



References

[Baquero and Preguiça, 2016] Baquero, C. and Preguiça, N. (2016).

Why logical clocks are easy.

Communications of the ACM, 59(4):43–47

DOI:10.1145/2890782

[Kshemkalyani and Singhal, 2011] Kshemkalyani, A. D. and Singhal, M. (2011).

Distributed Computing: Principles, Algorithms, and Systems.

Cambridge University Press

<https://www.cambridge.org/us/universitypress/subjects/engineering/communications-and-signal-processing/distributed-computing-principles-algorithms-and-systems>

[Lamport, 1978] Lamport, L. (1978).

Time, clocks, and the ordering of events in a distributed system.

Communications of the ACM, 21(7):558–565

DOI:10.1145/359545.359563

[Tanenbaum and van Steen, 2017] Tanenbaum, A. S. and van Steen, M. (2017).

Distributed Systems. Principles and Paradigms.

Pearson Prentice Hall, 3rd edition

<https://www.distributed-systems.net/index.php/books/ds3/>

